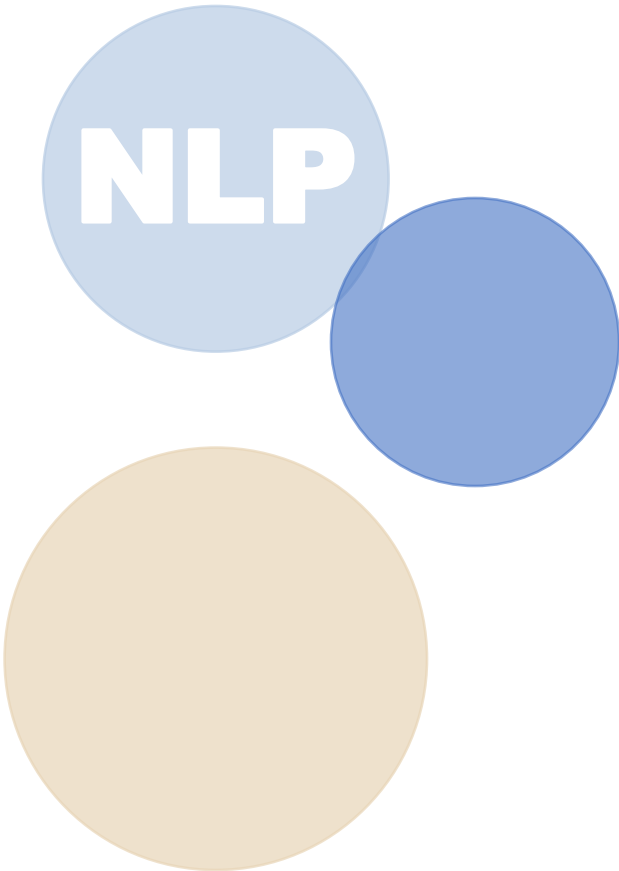


# [2주차] 텍스트 벡터화 기법

자연어의 수치화 및 Vector Space Model (VSM)



NLP

# Presentation

## Index

### 01 텍스트 전처리와 토큰화

- 단어·형태소·서브워드 토큰화
- BPE 알고리즘 원리
- 정규화·불용어 처리

### 03 TF-IDF 수학적 원리

- TF 변형 & IDF 유도
- Smoothing & 정규화
- 키워드 추출 응용

### 02 Bag-of-Words & DTM

- BoW 모델 수학적 정의
- Document-Term Matrix
- 희소 행렬·n-gram

### 04 VSM & 코사인 유사도

- Vector Space Model
- 코사인·자카드 유사도
- 미니 검색엔진 실습

# 01

## 텍스트 전처리와 토큰화(Tokenization)

Text Preprocessing & Tokenization

## Week 1 Recap & 동기(Motivation)

- 지난주: NLP 패러다임 변화 (Rule-based → Statistical → Deep Learning)
- 딥러닝(Transformer 등) 이전, 컴퓨터가 텍스트를 이해하려면 수학적 벡터 공간으로의 매핑이 필수
- 오늘 배울 DTM과 TF-IDF는 현대 임베딩의 근간이 되는 '원본 차원 공간'을 정의

## 텍스트 데이터의 이산적 특성 (Discrete Nature)

- 이미지·음성: 연속적(Continuous) 신호 ↔ 텍스트: 불연속적인 심볼(Symbol)의 집합
- 문제 정의: 이산 기호 집합(Vocabulary,  $V$ )을 정의하고, 문서( $d$ )를  $|V|$  차원 벡터 공간  $\mathbb{R}^{|V|}$ 에 투영
- 이 과정의 핵심 첫 단계가 바로 토큰화(Tokenization)

## 토큰화(Tokenization) 정형화

정의: 문자열  $S$ 를 의미 최소 단위 시퀀스  $T = [t_1, t_2, \dots, t_n]$ 으로 분할하는 함수  $f: S \rightarrow T$

## 단어 수준 토큰화 (Word Tokenization)

- 공백/구두점 기준 분리: `split(' ')`가 가장 단순한 방법
- 실제로는 구두점(Punctuation), 접어(Clitics: `I'm` → `I` + `'m`), 특수문자 처리가 필요
- 정규표현식(RegEx) 기반 Tokenizer로 보다 정교한 처리 가능

## OOV (Out-Of-Vocabulary) 문제

학습 데이터(Corpus)에 없는 단어가 테스트 데이터에 등장하면 처리 불가 → `<UNK>` 토큰화

- 단어장 크기 폭발: `run`, `runs`, `running` 등 변형된 단어 모두 독립 토큰으로 취급 시  $|V|$ 가 기하급수적으로 커짐
- 희소성(Sparsity) 문제: 대부분의 DTM 원소가 0이 되어 학습 효율 저하
- 해결 방향: 형태소 분석(한국어) 또는 서브워드 토큰화로 단어장 크기를 제어

## 형태소 분석 (Morphological Analysis) — 한국어의 특수성

- 한국어: 교착어(Agglutinative Language) — 어근(語根)에 접사(接辭)가 결합해 의미·문법 기능 부여
- '사과가', '사과를' → '사과(명사) + 가/를(조사)' 로 분리해야 어휘 폭발 방지
- 영어에는 없는 조사, 어미, 접두·접미사가 수만 가지 → 형태소 분석기 필수

### KoNLPy 형태소 분석기 비교

| 분석기     | 내부 모델     | 속도    | 정확도 | 비고           |
|---------|-----------|-------|-----|--------------|
| Okt     | 규칙+사전     | 보통    | 중   | 사용 편의성 높음    |
| Kkma    | CRF 기반    | 느림    | 높음  | 상세 분석 가능     |
| Mecab   | CRF (C++) | 매우 빠름 | 높음  | 실무 추천        |
| Komoran | 규칙+CRF    | 보통    | 높음  | 공백 없는 텍스트 강점 |

#### 내부 동작 원리

형태소 분석기는 단순 사전 매칭이 아닌, HMM(은닉 마르코프 모델)이나 CRF(조건부 무작위장)로 문맥상 가장 확률이 높은 형태소 시퀀스를 추론합니다.

## 현대 NLP의 표준: 서브워드 토큰화 (Subword Tokenization)

- 단어(Word) 단위의 OOV 문제 + 글자(Character) 단위의 의미 파악 어려움 → 두 장점을 결합
- 핵심 아이디어: '의미 있는 단위는 합치고, 처음 보는 단어는 잘게 쪼갬다'
- "unbelievable" → ["un", "##believ", "##able"] (WordPiece 예시)

### 주요 서브워드 알고리즘 비교

| BPE<br>(Byte Pair Encoding)                            | WordPiece                               | Unigram<br>(SentencePiece)                             |
|--------------------------------------------------------|-----------------------------------------|--------------------------------------------------------|
| 가장 빈번한 문자 쌍 반복 병합<br>GPT, LLaMA 등 사용<br>데이터 압축 알고리즘 지원 | BPE와 유사하나 최대 우도<br>기준으로 병합<br>BERT에서 사용 | Unigram LM 기반 확률적 분할<br>가장 유연한 방식<br>T5, ALBERT 등에서 사용 |

### 의의

OOV 거의 완전 해결 (최악의 경우 문자 단위) + GPT·LLaMA 등 모든 최신 LLM이 BPE 또는 그 변형을 Tokenizer로 채택

## BPE (Byte Pair Encoding) 알고리즘 심층 분석 (1)

- 원래 데이터 압축 알고리즘 → Sennrich et al. (2016) 에서 NLP 서브워드 토큰화에 도입
- Step 1: 모든 단어를 글자(Character) 단위로 분리 후 빈도수 측정
- Step 2: 가장 높은 빈도로 연속 등장하는 글자 쌍(Pair)을 병합(Merge) → 새 토큰으로 V에 추가
- Step 3: 지정된 Vocab Size에 도달할 때까지 Step 2를 반복 — 이 병합 규칙(Merge Rules)이 모델의 자산

## BPE 알고리즘 의사코드 (Pseudocode)

초기화: vocab = 글자 단위 단어장

```
while len(vocab) < target_vocab_size:
    pair_freq = count_pairs(corpus, vocab)
    best_pair = argmax(pair_freq)
    vocab = merge(vocab, best_pair)
    corpus = apply_merge(corpus, best_pair)

return vocab, merge_rules
```



## BPE 알고리즘 심층 분석 (2) — 작동 예시

Corpus: {"low":5, "lower":2, "newest":6, "widest":3}

| 단계     | 상위 빈도 쌍        | 병합 결과  | 처리 후 어휘                    |
|--------|----------------|--------|----------------------------|
| 초기     | -              | -      | l,o,w,e,r,n,s,t,i,d        |
| Step 1 | ('e','s') = 9  | es 생성  | l,o,w,e,r,n,es,t,i,d       |
| Step 2 | ('es','t') = 9 | est 생성 | l,o,w,e,r,n,est,i,d        |
| Step 3 | ('l','o') = 7  | lo 생성  | lo,w,e,r,n,est,i,d         |
| Step 4 | ('lo','w') = 7 | low 생성 | low,e,r,n,est,i,d (low 완성) |

## 서브워드 토큰화의 의미

- OOV 문제 거의 완벽 해결 (최악의 경우 문자 단위로 쪼개어 표현 가능)
- GPT, LLaMA 등 모든 최신 거대 언어 모델(LLM)이 BPE 또는 그 변형을 기본 Tokenizer로 채택
- Hugging Face tokenizers 라이브러리로 사용자 정의 BPE 모델 학습 가능

## 정규화 (Normalization) 및 정제 기법

### 대소문자 통일 (Lowercasing)

Apple → apple / NLP → nlp

### 불용어 제거 (Stopwords)

'the', 'a', '그리고', '은/는/이/가' 등 분석에 불필요한 단어 제거

### 표제어 추출 (Lemmatization)

running → run, better → good (문법적 원형 복원)

### 어간 추출 (Stemming)

running → run, studies → studi (규칙적 접미사 제거, 빠르지만 부정확)

## 전처리 파이프라인 완성



## [실습] Tokenizer 훈련시키기 — Hugging Face tokenizers

- Hugging Face tokenizers 라이브러리로 사용자 정의 BPE 모델을 처음부터 학습 가능
- fit() → encode() → decode() 인터페이스로 간편하게 사용
- 학습된 vocab.json과 merges.txt 파일이 모델의 Tokenizer 자산

```
from tokenizers import Tokenizer, models, trainers, pre_tokenizers
```

```
# 1. BPE 모델 초기화
```

```
tokenizer = Tokenizer(models.BPE())
```

```
tokenizer.pre_tokenizer = pre_tokenizers.Whitespace()
```

```
# 2. 학습 설정
```

```
trainer = trainers.BpeTrainer(
```

```
    vocab_size=5000,
```

```
    special_tokens=["<UNK>", "<PAD>", "<BOS>", "<EOS>"]
```

```
)
```

```
# 3. 텍스트 데이터로 학습
```

```
tokenizer.train(files=["corpus.txt"], trainer=trainer)
```

```
# 4. 저장 및 사용
```

```
tokenizer.save("my_tokenizer.json")
```

```
encoded = tokenizer.encode("자연어처리는 재미있다")
```

```
print(encoded.tokens)    # ['자연어', '##처리', '##는', '재미있다']
```

## One-Hot Encoding — 텍스트의 이산 공간 매핑

- 수학적 정의: 단어장 크기  $|V|$  일 때,  $i$  번째 단어  $\rightarrow i$  번째 원소만 1, 나머지 0인 벡터  $v \in \{0,1\}^{|V|}$
- 예)  $V = \{\text{고양이}, \text{강아지}, \text{새}\}$ , '고양이'  $\rightarrow [1, 0, 0]$ , '강아지'  $\rightarrow [0, 1, 0]$
- 장점: 구현이 매우 단순 / 단점: 차원이  $|V|$  만큼 폭발, 벡터 간 내적 항상 0 (의미 유사도 파악 불가)

## One-Hot 벡터의 치명적 한계 — '차원의 저주'

벡터 간 직교(Orthogonal): 모든 단어 쌍의 내적 = 0  $\rightarrow$  '고양이'와 '강아지'의 유사도 = '고양이'와 '자동차'의 유사도

## Part 1 정리 & 다음 단계로

- 토큰화  $\rightarrow$  정수 인덱스 부여(Integer Encoding)  $\rightarrow$  One-Hot  $\rightarrow$  BoW/DTM
- One-Hot의 한계를 극복하기 위해 '빈도' 정보를 활용하는 BoW 모델로 발전
- 2부에서: Bag-of-Words 모델과 Document-Term Matrix를 수학적으로 이해

# 02

## 수치화 기법

### Bag-of-Words & DTM

Text Vectorization

## Bag-of-Words (BoW) 가설

"문서는 단어들의 집합일 뿐, 순서(Sequence)와 문맥(Context)은 무시한다. 오직 단어의 출현 빈도만이 문서의 주제를 결정한다"

## BoW의 수학적 표현

- 문서  $d$ 에 대한 BoW 벡터:  $v_d = [f_1, f_2, \dots, f_V]$
- 여기서  $f_i$ 는 단어  $w_i$ 가 문서  $d$ 에 등장한 횟수 (빈도수, Term Frequency)
- 예)  $D = \text{'고양이 고양이 강아지'}$ ,  $V = \{\text{고양이:0, 강아지:1, 새:2}\} \rightarrow v_d = [2, 1, 0]$

## 직관적 예시

문서1: '인공지능 딥러닝 인공지능'

→ [인공지능:2, 딥러닝:1, 자연어:0]

문서2: '자연어 처리 딥러닝 자연어'

→ [인공지능:0, 딥러닝:1, 자연어:2]

## DTM (Document-Term Matrix) 구성

- 코퍼스  $D = \{d_1, d_2, \dots, d_N\}$ 을 행(Row), 단어장  $V$ 를 열(Column)로 하는  $N \times |V|$  행렬
- 각 원소  $x_{i,j}$  = 문서  $i$ 에 단어  $j$ 가 등장한 빈도수
- `sklearn.feature_extraction.text.CountVectorizer` 로 쉽게 구성 가능

|     | 인공지능 | 딥러닝 | 자연어 | 학습 | 모델 |
|-----|------|-----|-----|----|----|
| 문서1 | 3    | 2   | 1   | 2  | 0  |
| 문서2 | 1    | 0   | 3   | 1  | 2  |
| 문서3 | 0    | 1   | 2   | 3  | 1  |

## 희소 행렬 (Sparse Matrix) 문제

- 실제 코퍼스에서 Sparsity(희소성)는 99% 이상 — 대부분의 원소가 0
- `numpy.array`로 저장 시 OOM(Out of Memory) 발생 가능
- 해결책: `scipy.sparse`의 CSR(Compressed Sparse Row) 포맷 사용 필수

## BoW / DTM의 한계 분석

### ① 어순(Sequence) 무시

"존은 제인을 사랑한다" vs "제인은 존을 사랑한다"  
→ 완전히 동일한 BoW 벡터로 매핑됨 (정보 손실)

### ② 빈도수의 역설

문장 길이에 비례해 값이 커지고, 'the', 'a', '그리고' 같은 범용 단어가 가장 큰 빈도수  
→ 핵심 주제어(Keyword) 가치 희석

### ③ 의미론적 유사성 파악 불가

'Car'와 'Automobile'은 의미가 같지만 BoW에서는 완전히 다른 차원 (직교 벡터)  
→ 유사도 = 0

## n-gram 기반 BoW — 부분적 보완책

- 어순 무시의 단점을 보완: 1개(Unigram)가 아닌 2개(Bigram), 3개(Trigram) 단위로 빈도 카운트
- 예) '자연어 처리' → Bigram 토큰 ('자연어','처리') 로 취급
- 단점:  $|V|$ 의 크기가 제곱, 세제곱으로 팽창 → Sparsity 문제 극대화
- CountVectorizer(ngram\_range=(1,2)) 로 Sklearn에서 Bigram BoW 구성 가능



## DTM 파이썬 실습 — CountVectorizer

- sklearn CountVectorizer의 fit\_transform 과정에서 내부적으로 딕셔너리(vocabulary\_)가 생성됨
- 반환값: CSR(Compressed Sparse Row) 희소 행렬 → .toarray()로 밀집 행렬 변환 가능
- vocabulary\_ 속성으로 단어 → 인덱스 매핑 확인 가능

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    '인공지능 딥러닝 인공지능',
    '자연어 처리 딥러닝 자연어',
    '딥러닝 학습 모델 학습'
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

print(vectorizer.vocabulary_)
# {'인공지능': 3, '딥러닝': 0, '자연어': 2, '처리': 4, '학습': 5, '모델': 1}

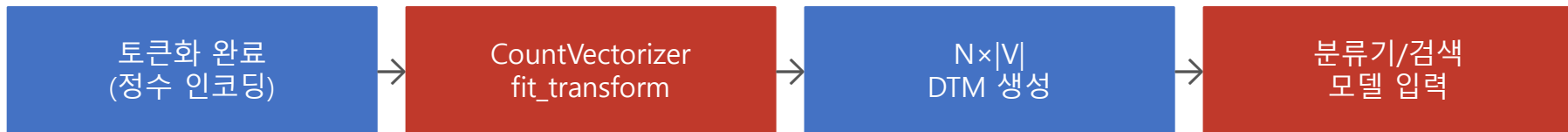
print(X.toarray())
# [[2 2 0 2 0 0]
#    [0 1 2 0 1 0]
#    [0 1 0 0 0 2]]
```

□ fit\_transform = 어휘 사전 구축(fit) + 행렬 변환(transform) 동시 수행

## 텍스트 마이닝에서의 DTM 활용

- DTM 행렬에 나이브 베이즈(Naive Bayes) 분류기를 적용하면 스팸 메일 분류에서 우수한 베이스라인 성능
- 로지스틱 회귀(Logistic Regression) + DTM으로도 감정 분석, 주제 분류 등 고전적 NLP 작업 수행 가능
- 현재도 간단한 텍스트 분류, 검색, 문서 클러스터링에 활발히 사용

## Part 2 정리



## 한계 및 다음 단계

DTM의 단순 빈도 가중치 → 범용 단어(the, 그리고)가 핵심어보다 높은 값을 가짐 → TF-IDF로 해결

3부에서: 단순 빈도 카운트의 한계를 극복하는 TF-IDF 가중치 기법을 학습합니다.

# 03

## 가중치 조정

### TF-IDF의 수학적 원리

TF-IDF Weighting

## TF-IDF 철학적 배경

"어떤 단어가 특정 문서에 자주 등장하면 중요하지만(TF ↑),  
그 단어가 코퍼스 내 모든 문서에 자주 등장한다면 범용적인 단어로서 중요하지 않다(IDF ↓)"

## TF (Term Frequency)의 다양한 변형(Variants)

### Boolean TF

단어 등장하면 1, 아니면 0

```
tf = 1 if f > 0 else 0
```

### Raw Count

단순 빈도수 (기본값)

```
tf = f_{t,d}
```

### Log-normalization

빈도 증가폭 감소 (로그 스케일)

```
tf = log(1 + f_{t,d})
```

### Double Normalization

길이 편향 방지 (0.5 기준)

```
tf = 0.5 + 0.5 × (f_{t,d} /  
max f_{t',d})
```

## 정보 이론 (Information Theory)과 IDF의 관계

- IDF는 우연히 발견된 수식이 아닌, 클로드 섀넌(Claude Shannon)의 정보 이론에 근거
- 단어  $t$ 가 등장할 확률  $P(t) = DF(t) / N$
- 이 단어가 주는 '자기 정보량(Self-Information)'  $= -\log P(t) = \log(N / DF(t))$
- → 희귀한 단어일수록 정보량이 크다는 직관과 일치

## IDF (Inverse Document Frequency) 수식 분석

$$IDF(t, D) = \log( N / |\{d \in D : t \in d\}| )$$

- $N$ : 전체 문서 수 / 분모: 단어  $t$ 가 포함된 문서의 수
- 모든 문서에 등장하는 단어(분모= $N$ )  $\rightarrow \log(1) = 0 \rightarrow IDF=0$  (완전히 무의미)
- 단 1개 문서에만 등장(분모=1)  $\rightarrow \log(N) \rightarrow$  매우 높은 IDF (중요 키워드 후보)

## Zero-Division & Smoothing (스무딩)

## TF-IDF 스코어 결합

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

- '곱셈' 결합: 두 조건(해당 문서 내 고빈도 AND 전체 문서 내 저빈도)을 동시에 만족할 때 스코어 극대화
- 한쪽이 0이면 전체 스코어 0 → 완전 범용 단어(IDF=0)는 자동으로 제거
- 두 조건이 모두 높을 때만 '그 문서의 특징어(Characteristic Term)'로 선택됨

## L2 Normalization (벡터 정규화)의 필수성

- 긴 문서: TF 값이 전반적으로 크기 때문에 TF-IDF 벡터의 크기(Norm)도 커짐
- 짧은 문서와 비교 시 불공평 → 벡터의 방향만 유지하고 크기를 1로 맞추는 L2 정규화 필요
- 정규화 공식:  $v_{\text{norm}} = v / \|v\|_2$
- sklearn TfidfVectorizer에는 L2 정규화가 기본값(norm='l2')으로 적용됨

## [수기 실습] TF-IDF 테이블 직접 계산하기 (1/3) — 미니 코퍼스 설정

미니 코퍼스 (N=3): D1='인공지능 학습 학습', D2='딥러닝 학습 인공지능', D3='자연어 처리 딥러닝'

## Step 1: Raw Count TF 행렬 구성

|    | 인공지능 | 딥러닝 | 자연어 | 학습 | 처리 |
|----|------|-----|-----|----|----|
| D1 | 1    | 0   | 0   | 2  | 0  |
| D2 | 1    | 1   | 0   | 1  | 0  |
| D3 | 0    | 1   | 1   | 0  | 1  |

## Step 2: DF(문서 빈도) 계산

| 단어           | 인공지능 | 딥러닝 | 자연어 | 학습 | 처리 |
|--------------|------|-----|-----|----|----|
| DF (등장 문서 수) | 2    | 2   | 1   | 2  | 1  |

## [수기 실습] TF-IDF 테이블 직접 계산하기 (2/3) — IDF 벡터 계산

IDF 공식:  $IDF(t) = \log(N / DF(t)) + 1$  ( $N=3$ , 자연로그 기준)

| 단어   | DF   | $\log(N/DF)$    | 계산            | IDF 값           |
|------|------|-----------------|---------------|-----------------|
| 인공지능 | DF=2 | $\log(3/2) + 1$ | $= 0.405 + 1$ | $\approx 1.405$ |
| 딥러닝  | DF=2 | $\log(3/2) + 1$ | $= 0.405 + 1$ | $\approx 1.405$ |
| 자연어  | DF=1 | $\log(3/1) + 1$ | $= 1.099 + 1$ | $\approx 2.099$ |
| 학습   | DF=2 | $\log(3/2) + 1$ | $= 0.405 + 1$ | $\approx 1.405$ |
| 처리   | DF=1 | $\log(3/1) + 1$ | $= 1.099 + 1$ | $\approx 2.099$ |

## 결론

- 자연어·처리 (DF=1):  $IDF \approx 2.099 \rightarrow$  특정 문서에만 나타나는 고유 단어, 더 높은 가중치
- 인공지능·딥러닝·학습 (DF=2):  $IDF \approx 1.405 \rightarrow$  여러 문서에 공통으로 나타나 상대적으로 낮은 가중치
- 만약  $DF=N$ (전체 문서)이면  $IDF = \log(1) + 1 = 1 \rightarrow$  최소값 부여



## [수기 실습] TF-IDF 테이블 직접 계산하기 (3/3) — 최종 행렬 도출

TF-IDF = TF × IDF (각 셀은 TF값 × 해당 열의 IDF값)

|    | 인공지능<br>(×1.405) | 딥러닝<br>(×1.405) | 자연어<br>(×2.099) | 학습<br>(×1.405) | 처리<br>(×2.099) |
|----|------------------|-----------------|-----------------|----------------|----------------|
| D1 | 1×1.405=1.405    | 0×1.405=0       | 0×2.099=0       | 2×1.405=2.810  | 0×2.099=0      |
| D2 | 1×1.405=1.405    | 1×1.405=1.405   | 0×2.099=0       | 1×1.405=1.405  | 0×2.099=0      |
| D3 | 0×1.405=0        | 1×1.405=1.405   | 1×2.099=2.099   | 0×1.405=0      | 1×2.099=2.099  |

## 해석

- D1의 키워드: '학습'(2.810) > '인공지능'(1.405) — D1을 대표하는 단어
- D3의 키워드: '자연어'(2.099), '처리'(2.099) — D3에만 나타나 높은 IDF 덕에 강조
- L2 정규화 후 각 문서 벡터의  $\|v\|_2 = 1$  이 되어 공정한 문서 간 비교 가능

## Scikit-learn TfidfVectorizer 파라미터 튜닝

```
max_df=0.85
```

전체 문서의 85% 이상에서 등장하는 단어 제외 (너무 범용적)

불용어 자동 제거 효과

```
min_df=2
```

2개 미만의 문서에만 등장하는 단어 제외 (너무 희귀)

노이즈 단어 제거 효과

```
sublinear_tf=True
```

TF에  $\log(1+TF)$  스케일 적용  
빈도 10배라고 중요도가 10배?

빈도 편향 완화

```
norm='l2'
```

기본값. 각 문서 벡터를 L2 정규화

문서 길이 편향 제거

```
ngram_range=(1,2)
```

Unigram + Bigram 동시 사용

어순 정보 부분 반영

## TF-IDF 행렬 시각화 — Heatmap

- Seaborn 히트맵으로 TF-IDF 가중치를 시각화하면, 각 문서별 핵심 단어가 진한 색으로 강조됨
- 뉴스 기사 분류 실험: '경제' 기사의 TF-IDF에서 '금리', '주식', 'GDP' 등이 붉게 표시

|      | 금리   | 주식   | GDP  | 국회   | 법안   | 선거   | 기후   | 탄소   | 에너지  |
|------|------|------|------|------|------|------|------|------|------|
| 경제기사 | 0.82 | 0.76 | 0.65 | 0.02 | 0.01 | 0.01 | 0.01 | 0.02 | 0.01 |
| 정치기사 | 0.01 | 0.02 | 0.03 | 0.79 | 0.71 | 0.83 | 0.01 | 0.02 | 0.01 |
| 환경기사 | 0.01 | 0.02 | 0.01 | 0.02 | 0.01 | 0.02 | 0.81 | 0.77 | 0.69 |

## 키워드 추출 (Keyword Extraction) 응용

- TF-IDF 행렬에서 각 문서별로 가장 스코어가 높은 Top-K 단어를 추출하면 자동 키워드 요약 가능
- 뉴스 기사 자동 태그, 문서 검색 색인, 추천 시스템 등에 활발히 활용

## TF-IDF 파이썬 실습 — TfidfVectorizer & 키워드 추출

```
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np

corpus = [
    '인공지능 딥러닝 학습 학습',
    '자연어 처리 딥러닝',
    '인공지능 모델 학습 자연어'
]

vectorizer = TfidfVectorizer(sublinear_tf=True, max_df=0.9, min_df=1)
X = vectorizer.fit_transform(corpus)

feature_names = vectorizer.get_feature_names_out()

# 문서별 Top-3 키워드 추출
for i, row in enumerate(X.toarray()):
    top_idx = np.argsort(row)[::-1][:3]
    keywords = [feature_names[j] for j in top_idx]
    print(f'문서{i+1} 키워드: {keywords}')

# 출력 결과:
# 문서1 키워드: ['학습', '인공지능', '딥러닝']
# 문서2 키워드: ['처리', '자연어', '딥러닝']
# 문서3 키워드: ['자연어', '인공지능', '모델']
```

# 04

## Vector Space Model

### 문서 유사도 & 검색

Document Similarity & IR

## Vector Space Model (벡터 공간 모델)

- 문서를 텍스트가 아닌 수학적 공간의 '좌표' 또는 '화살표(벡터)'로 바라보는 패러다임
- 질의(Query) 역시 하나의 '짧은 문서'로 벡터화 → 질의 벡터와 가장 가까운 문서 벡터를 찾는 것이 IR의 핵심
- 벡터 공간에서의 '거리(Distance)' 또는 '방향의 유사성'이 문서의 관련성을 결정

## 벡터 간 거리를 측정하는 방법들

| L1 Distance<br>(Manhattan) | L2 Distance<br>(Euclidean)  | Cosine<br>Similarity              | Jaccard<br>Similarity     |
|----------------------------|-----------------------------|-----------------------------------|---------------------------|
| $\sum  A_i - B_i $         | $\sqrt{\sum (A_i - B_i)^2}$ | $A \cdot B / (\ A\  \cdot \ B\ )$ | $ A \cap B  /  A \cup B $ |
| 각 차원의 절댓값 차이 합산            | 직선 거리. 텍스트에 부적합             | 텍스트 표준. 방향만 비교                    | 집합 기반. 짧은 텍스트 유용          |

## 왜 텍스트 데이터에 유클리드 거리는 부적합한가?

유클리드 거리:  $d(A, B) = \sqrt{\sum (A_i - B_i)^2}$

### 반례 시나리오

문서 A: '인공지능 딥러닝' (단어 2개)

문서 B: '인공지능 딥러닝 인공지능 딥러닝' (문서 A를 2번 복사, 단어 4개)

→ 두 문서의 주제는 완전히 동일하지만, 유클리드 거리는 벡터 크기 차이 때문에 멀어짐!

## 코사인 유사도는 이 문제를 어떻게 해결하는가?

- 코사인 유사도는 벡터의 크기(Magnitude)를 무시하고 오직 '방향(Direction)'만 비교
- 문서 A와 문서 B는 크기는 다르지만 벡터의 방향이 완전히 같음 → 코사인 유사도 = 1.0
- 즉, '같은 내용을 더 많이 반복한 문서'와 '원본 문서'는 동일한 관련성을 가진다고 판단

## L2 정규화와 코사인 내적의 관계

TF-IDF 벡터가 L2 정규화된 경우:  $\|A\| = \|B\| = 1 \rightarrow \cos(\theta) = A \cdot B$  (단순 내적으로 연산 속도 극대화)

## 코사인 유사도 (Cosine Similarity)의 수학적 유도

- 두 벡터 A, B 사이의 내적(Dot Product) 공식:  $A \cdot B = \|A\| \cdot \|B\| \cdot \cos(\theta)$
- 양변을  $\|A\| \cdot \|B\|$ 로 나누면:  $\cos(\theta) = A \cdot B / (\|A\| \cdot \|B\|)$

$$\cos\_sim(A, B) = (A \cdot B) / (\|A\|_2 \times \|B\|_2) = \frac{\sum (A_i \cdot B_i)}{\sqrt{(\sum A_i^2)} \times \sqrt{(\sum B_i^2)}}$$

## 코사인 유사도의 범위와 텍스트 도메인 특성

- 수학적 범위:  $-1 \leq \cos(\theta) \leq 1$
- TF-IDF 벡터의 모든 원소는 0 이상  $\rightarrow$  텍스트 간 코사인 유사도는 항상  $0 \leq sim \leq 1$
- $sim = 0$ : 완전히 무관한 문서 (공통 단어 없음, 직교)
- $sim = 1$ : 완전히 동일한 방향의 문서 (같은 주제)

## 자카드 유사도 (Jaccard Similarity) 보충

$J(A, B) = |A \cap B| / |A \cup B|$  — 단어 집합의 교집합 비율. 빈도 무시, 짧은 텍스트(제목 유사도 등)에 효과적



## [프로젝트 실습] 미니 검색엔진 만들기 (1/3) — 데이터 준비

- 1만 건의 뉴스 기사 데이터셋 로드 (예: KLUE 벤치마크 뉴스 데이터셋)
- TfidfVectorizer를 통해  $10000 \times V$  차원의 TF-IDF 행렬 생성
- Fit 단계에서 어휘 사전 구축, Transform 단계에서 각 기사를 TF-IDF 벡터로 변환

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

# 데이터 로드
df = pd.read_csv('news_articles.csv') # 뉴스 기사 1만 건
corpus = df['content'].tolist()
titles = df['title'].tolist()

# TF-IDF 행렬 생성
vectorizer = TfidfVectorizer(
    max_df=0.85, min_df=2,
    ngram_range=(1,2),
    sublinear_tf=True
)
X = vectorizer.fit_transform(corpus)
print(f'행렬 크기: {X.shape}') # (10000, 어휘크기)
print(f'희소도: {1 - X.nnz / (X.shape[0]*X.shape[1]):.4f}')
```

## [프로젝트 실습] 미니 검색엔진 만들기 (2/3) — 질의 벡터 생성

- 사용자 검색어 '인공지능 발전'을 입력
- 학습된 vectorizer의 transform 메서드로 질의를 Query Vector ( $1 \times V$  행렬)로 변환
- fit 단계에서 본 적 없는 단어는 무시(ignore), 학습 어휘에 있는 단어만 벡터화

```
from sklearn.metrics.pairwise import cosine_similarity, linear_kernel
import numpy as np

# 검색어 입력
query = '인공지능 발전'

# Query를 벡터로 변환 (fit 없이 transform만)
query_vec = vectorizer.transform([query]) # shape: (1, V)

# 전체 10000개 뉴스와 코사인 유사도 일괄 계산
# (L2 정규화 완료 시 linear_kernel이 cosine_similarity보다 빠름)
scores = linear_kernel(query_vec, X).flatten()

print(f'유사도 벡터 크기: {scores.shape}') # (10000,)
print(f'최고 유사도: {scores.max():.4f}')
```

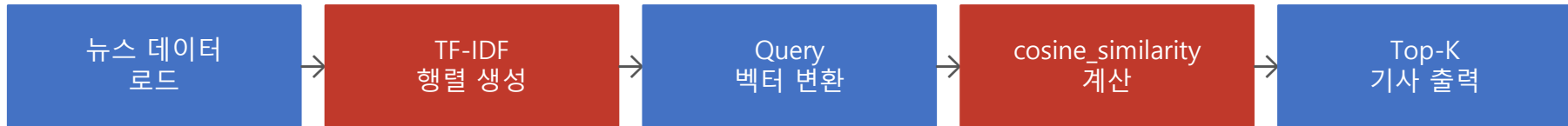
## [프로젝트 실습] 미니 검색엔진 만들기 (3/3) — Top-K 결과 출력

```
# 유사도 Top-5 기사 인덱스 추출
top_k = 5
top_indices = np.argsort(scores)[:,::-1][:top_k]

print(f'\n[검색어: "{query}"] 관련 기사 Top-{top_k}\n')
for rank, idx in enumerate(top_indices, 1):
    print(f'{rank}위 [유사도: {scores[idx]:.4f}] {titles[idx]}')

# 출력 예시:
# 1위 [유사도: 0.8231] 인공지능 기술 발전으로 변화하는 산업 지형
# 2위 [유사도: 0.7943] AI 발전 속도, 인류의 미래는?
# 3위 [유사도: 0.7612] 딥러닝과 인공지능의 최신 연구 동향
```

## 전체 검색엔진 파이프라인



# 05

## 한계점 고찰 및 향후 학습 방향

Limitations & Next Steps

## Count 기반 모델(TF-IDF)의 근본적 한계

### ① 어휘 불일치 (Vocabulary Mismatch)

'Car'와 'Automobile'은 의미가 같지만 TF-IDF에서는 완전히 다른 차원의 직교 벡터  
→ 코사인 유사도 = 0 (의미론적 유사성 파악 불가)

### ② 문맥(Context) 정보 손실

'밥을 먹다'와 '먹을 밥'은 의미가 다르지만 BoW에서는 동일한 벡터  
→ 어순, 의존 관계 등 문맥 정보 완전 무시

### ③ 고차원 희소 벡터 (High-dim Sparse Vector)

단어장 크기  $|V|$ 가 수만~수십만 → 대부분의 원소가 0인 희소 벡터  
→ 메모리 비효율, 학습 어려움, 차원의 저주

## Sparse Vector vs Dense Vector 비교

| Sparse Vector (TF-IDF) |                                         | Dense Vector (Word Embedding)          |  |
|------------------------|-----------------------------------------|----------------------------------------|--|
| 차원                     | $ V  \approx \text{수만} \sim \text{수십만}$ | $d \approx 100 \sim 1000$ (고정)         |  |
| 원소 값                   | 대부분 0 (희소)                              | 모두 실수 (밀집)                             |  |
| 의미 포착                  | 단어 빈도만 반영                               | 단어 의미·문맥 학습                            |  |
| 유사어 처리                 | $\text{Car} \neq \text{Automobile}$     | $\text{Car} \approx \text{Automobile}$ |  |
| 학습 필요                  | 없음 (통계 기반)                              | 대규모 말뭉치 학습 필요                          |  |

Dense Vector는 TF-IDF의 한계를 극복하는 Word Embedding 기법(Word2Vec 등)으로 생성됩니다

## 2주차 전체 학습 내용 요약

| 01 Tokenization                                      | 02 BoW & DTM                                             | 03 TF-IDF                                            | 04 VSM & 코사인 유사도                                                              |
|------------------------------------------------------|----------------------------------------------------------|------------------------------------------------------|-------------------------------------------------------------------------------|
| 문자열 → 토큰 시퀀스. 단어/형태소/서브워드(BPE) 방식. OOV 문제 → 서브워드로 해결 | 단어 빈도 기반 문서 벡터화. $N \times  V $ 희소 행렬. 어순 무시, 범용어 가중치 문제 | TF×IDF 가중치. 정보 이론 기반 IDF. L2 정규화. TfidfVectorizer 활용 | 문서를 벡터 공간의 점으로. $\cos(\theta) = A \cdot B / (\ A\  \cdot \ B\ )$ . 미니 검색엔진 구현 |

전체 흐름: Raw Text → Tokenization → Integer Encoding → TF-IDF Matrix → Cosine Similarity → 검색/분류

## 핵심 파이썬 라이브러리 정리

KoNLPy

sklearn.CountVectorizer

sklearn.TfidfVectorizer

sklearn.cosine\_similarity

한국어 형태소 분석기 (Okt, Mecab, Kkma 등)

BoW / DTM 생성

TF-IDF 행렬 생성 + L2 정규화

코사인 유사도 계산

## 다음 주 예고 — 3주차: Word Embedding (워드 임베딩)

TF-IDF의 고차원 희소(Sparse) 벡터 한계를 극복하고, 단어를 저차원 밀집(Dense) 벡터 공간에 표현하여 '의미(Semantics)'를 학습하는 Word2Vec을 3주차에 학습합니다.

### CBOW (Continuous Bag-of-Words)

주변 단어들(Context)로 중심 단어(Target)를 예측  
예) ' \_ 학습은 재미있다' → '딥러닝' 예측

### Skip-gram

중심 단어(Target)로 주변 단어(Context)를 예측  
예) '딥러닝' → '학습', '재미있다' 예측  
소규모 데이터에서 CBOW보다 우수

## Word2Vec의 혁신적 특성

- 의미론적 연산: '왕' - '남자' + '여자' = '여왕' (벡터 공간에서 실제로 성립)
- 유사 단어 군집: 비슷한 의미의 단어들이 벡터 공간에서 가까이 위치
- 고정 차원(예: 300차원) Dense 벡터 → 메모리 효율적, 딥러닝 입력으로 바로 활용 가능



## 정보 이론 심화 — 엔트로피(Entropy)와 TF-IDF의 관계

- 새넨 엔트로피  $H(X) = -\sum P(x) \log P(x)$ : 불확실성의 척도
- 어떤 단어가 예측 가능할수록(자주 등장) 정보량 낮음  $\rightarrow$  IDF가 낮아지는 것과 일치
- 역으로, 희귀 단어는 높은 정보량(Self-Information)  $= \log(1/P) = -\log P$  를 지님

## 정보량 계산 예시

```
import math

N = 1000 # 전체 문서 수

# '그리고'는 800개 문서에 등장  $\rightarrow$  흔한 단어
df_common = 800
idf_common = math.log(N / df_common) + 1
print(f'그리고 IDF: {idf_common:.4f}') #  $\rightarrow$  1.2231 (낮음)

# '퀀텀컴퓨팅'은 5개 문서에만 등장  $\rightarrow$  희귀 단어
df_rare = 5
idf_rare = math.log(N / df_rare) + 1
print(f'퀀텀컴퓨팅 IDF: {idf_rare:.4f}') #  $\rightarrow$  6.2983 (높음)
```

## 한국어 NLP 특수 이슈 — 전처리 주의사항

- 자모 분리 문제: 완성형 한글과 자모 단위 한글이 혼재할 수 있음 (유니코드 정규화 NFC 적용)
- 신조어/줄임말: '갑분싸', 'ㅋㅋ', '헐' 등 사전에 없는 표현 → 서브워드 토큰화로 어느 정도 대응
- 복합명사 처리: '딥러닝기반자연어처리' → '딥러닝 기반 자연어 처리'로 분리 필요

## KoNLPy 실습 코드 — Mecab 형태소 분석

```
from konlpy.tag import Mecab

mecab = Mecab()
text = '자연어처리는 인공지능의 핵심 분야입니다'

# 형태소 분석
morphs = mecab.morphs(text)
print('형태소:', morphs)
# ['자연어', '처리', '는', '인공지능', '의', '핵심', '분야', '입니다']

# 명사만 추출
nouns = mecab.nouns(text)
print('명사:', nouns)
# ['자연어', '처리', '인공지능', '핵심', '분야']
```

## 종합 실습 — 한국어 TF-IDF 문서 유사도 전체 파이프라인

```
from konlpy.tag import Mecab
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 1. 형태소 분석기 로드
mecab = Mecab()

# 2. 문서 전처리 (명사만 추출)
documents = ['인공지능 딥러닝 학습 모델', '자연어처리 텍스트 분석', '딥러닝 자연어처리 응용']
tokenized = [' '.join(mecab.nouns(doc)) for doc in documents]

# 3. TF-IDF 행렬 생성
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(tokenized)

# 4. 코사인 유사도 행렬
sim_matrix = cosine_similarity(X)
print('유사도 행렬:')
print(sim_matrix.round(3))
```

## 텍스트 임베딩 기법의 진화 (발전 로드맵)



2주차에서는 BoW/TF-IDF(통계 기반) 학습 → 3주차부터 Word2Vec(신경망 기반) 임베딩으로 발전

## 왜 임베딩의 역사를 이해해야 하는가?

- 각 시대별 기법의 한계가 다음 기법의 탄생 동기 → 발전 방향 이해에 필수
- TF-IDF도 여전히 키워드 추출, 빠른 검색 색인, 경량 분류기 등에서 실무 활용 중
- BERT, GPT도 결국 내부적으로 임베딩 레이어를 통해 토큰을 Dense 벡터로 변환

## 과제(Assignment) 및 추가 학습 자료

### □ 이번 주 과제

- 자신이 선택한 한국어 뉴스 기사 10개 이상을 수집하여 KoNLPy로 전처리
- TfidfVectorizer로 TF-IDF 행렬을 구성하고, 가장 중요한 Top-5 키워드를 기사별로 추출
- 두 기사 간 코사인 유사도 행렬을 계산하고, 유사도가 가장 높은 쌍과 그 이유를 보고서로 제출

### □ 추천 학습 자료

- Jurafsky & Martin, 'Speech and Language Processing' (3rd ed.) — Chapter 6: Vector Semantics
- 딥 러닝을 이용한 자연어 처리 입문 (WikiDocs) — <https://wikidocs.net/book/2155>
- Scikit-learn 공식 문서 — TfidfVectorizer API Reference
- Hugging Face Tokenizers 라이브러리 공식 문서 — 사용자 정의 BPE 학습 예제

제출: 다음 수업 전까지 / 형식: Jupyter Notebook (.ipynb) + 결과 PDF

## TF-IDF의 실전 산업 응용 사례

### □ 검색엔진

사용자 질의와 문서 간 TF-IDF 코사인 유사도로 관련 문서 랭킹 (Google 초기 알고리즘)

### □ 스팸 필터링

'무료', '당첨', '클릭' 등의 단어 TF-IDF 가중치 + 나이브 베이즈 분류기 조합

### □ 뉴스 추천

사용자가 읽은 기사와 코사인 유사도가 높은 기사를 추천 (콘텐츠 기반 필터링)

### □ 표절 탐지

두 문서 간 TF-IDF 코사인 유사도가 임계값 이상이면 유사 문서로 판정

## 자주 묻는 질문 (FAQ)

### Q. TF-IDF와 BoW의 차이는?

BoW는 단순 빈도(TF)만 사용. TF-IDF는 IDF로 범용 단어의 가중치를 낮추어 더 의미 있는 단어를 강조합니다.

### Q. L2 정규화는 왜 해야 하나요?

긴 문서는 TF-IDF 값이 전반적으로 크므로 짧은 문서와 공정하게 비교하려면 벡터 크기를 1로 맞추는 정규화가 필요합니다.

### Q. 코사인 유사도 값이 0.5이면 '반쯤 유사'한 건가요?

아닙니다. 코사인 유사도는 선형 척도가 아닙니다. 0.9 이상이면 매우 유사, 0.3 이하면 관련성 낮음으로 보는 것이 일반적입니다.

### Q. `max_df=0.85`의 의미는?

전체 문서의 85% 이상에서 나타나는 단어는 불용어로 간주해 어휘에서 제외합니다. 이 값을 낮추면 더 많은 단어가 제외됩니다.

### 3주차 예습 가이드 — Word2Vec 사전 지식

#### 다음 수업까지 이해하고 오면 좋은 개념들

- 신경망 순전파(Forward Propagation) / 역전파(Backpropagation) 기초 (Machine Learning 수업 내용 복습)
- 소프트맥스(Softmax) 함수 — 확률 분포를 출력하는 활성화 함수
- 원-핫 인코딩(One-Hot Encoding)이 Word2Vec 입력으로 어떻게 사용되는지
- 벡터의 내적(Dot Product)과 행렬 곱(Matrix Multiplication) — 임베딩 레이어 연산의 핵심

#### Word2Vec 핵심 개념 미리보기

##### 분산 의미 가설 (Distributional Hypothesis)

"비슷한 문맥에서 등장하는 단어들은 비슷한 의미를 가진다"

예시: '강아지는 뼈를 좋아한다', '고양이는 생선을 좋아한다'

→ '강아지'와 '고양이'는 유사한 문맥에 등장 → 벡터 공간에서 가까운 위치



## 자연어처리 전체 학기 로드맵 (수업 포지셔닝)

|             |                 |                   |                    |                    |               |                   |      |
|-------------|-----------------|-------------------|--------------------|--------------------|---------------|-------------------|------|
| 1주          | 2주              | 3주                | 4주                 | 5주                 | 6주            | 7주                | 8주   |
| NLP 개요      | 텍스트 벡터화<br>(현재) | Word<br>Embedding | Transformer<br>구조  | Hugging Face<br>실습 | 한국어<br>전처리    | Fine-tuning<br>기초 | 중간고사 |
| 9주          | 10주             | 11주               | 12주                | 13주                | 14주           | 15주               |      |
| 오픈소스<br>LLM | LangChain<br>기초 | RAG 기초            | LangGraph<br>& MCP | 프로젝트<br>개발(1)      | 프로젝트<br>개발(2) | 기말고사              |      |

이론 기반(1~4주) → Hugging Face 실습(5~7주) → 중간고사 → LLM·RAG 응용(9~12주) → 프로젝트(13~14주)

## 2주차 위치의 중요성

- 이번 주(2주차)에 배운 텍스트 벡터화는 이후 모든 임베딩 기법의 출발점이자 비교 기준
- Word2Vec(3주) → Transformer(4주) → 사전학습 모델(5~7주)로 이어지는 임베딩 발전 흐름의 첫 단계



# Thank you!

- 김정순 -

Email : [js.kim@deu.ac.kr](mailto:js.kim@deu.ac.kr)